

SEEDS CLUSTERING

Machine Learning Pipeline — Unsupervised Pattern Discovery

KMeans | REST API | config.yaml | run_pipeline.py | Libreria Riutilizzabile

210

Campioni

7

Feature

3

Varietà di grano

k=3

Cluster ottimali

Indice

Struttura della presentazione

01

Il Problema

UCI Seeds e obiettivo clustering

02

Architettura

Docker, REST API, config.yaml

03

Libreria lib/

Componenti generici riutilizzabili

04

Data Pipeline

Sequenza completa dei transformer

05

Algoritmo K-Means

Lloyd, k-means++, convergenza

06

Scelta di K

Elbow method e Silhouette score

07

Metriche Clustering

ARI, NMI, Purity

08

Config & Pipeline

config.yaml, run_pipeline.py, /plots/

9

Risultati

Performance su Seeds

10

Visualizzazioni

PCA 2D, profili, pairplot

11

Sviluppi futuri

Clustering gerarchico, DBSCAN

Il Problema — UCI Seeds

Unsupervised clustering: scoprire varietà di grano senza usare le etichette

Obiettivo

Identificare automaticamente le 3 varietà di grano (Kama, Rosa, Canadian) dalle misure geometriche, senza usare le etichette durante il training.

Dataset UCI Seeds — file locale (.txt)

210 campioni totali (70 per varietà, bilanciato)

7 feature numeriche continue, nessun valore mancante

Target (Class) caricato ma NON usato nel training

Valutazione: Silhouette + ARI/NMI/Purity post-fit

7 Feature (tutte continue)

area, perimeter, compactness

kernel_length, kernel_width

asymmetry_coef, groove_length

Le 3 varietà di grano (dati derivati dalla target)

Kama

70 campioni

Semi compatti e simmetrici. Forma arrotondata, compactness alta.

Rosa

70 campioni

Semi piu' grandi e allungati. Area e perimetro piu' elevati.

Canadian

70 campioni

Semi piu' piccoli e asimmetrici. Groove length caratteristica.

Architettura del Progetto

Docker | REST API | config.yaml | Libreria embedded

```
Seeds-ML/  
├── data/  
│   └── seeds_dataset.txt  
├── lib/  
│   ├── loader.py  
│   ├── eda.py  
│   ├── cleaner.py  
│   ├── encoder.py  
│   ├── kmeans.py  
│   ├── plots.py  
│   └── api/  
│       ├── state.py  
│       ├── loader.py  
│       ├── eda.py  
│       ├── cleaner.py  
│       ├── encoder.py  
│       ├── kmeans.py  
│       └── plots.py  
├── main.py  
├── app.py  
├── run_pipeline.py  
├── config.yaml  
├── Dockerfile  
└── requirements.txt
```

REST API — Flask

POST /loader/load → carica .txt locale
POST /encoder/fit_transform → scala feature
POST /kmeans/search → elbow + silhouette
POST /kmeans/fit → fit con k scelto (k=3)
GET /kmeans/compare → ARI, NMI, purity
GET /plots/ → lista PNG generati
GET /plots/<file> → scarica PNG

config.yaml — punto di controllo

```
loader:  
  csv_path: seeds_dataset.txt  
  target_col: Class  
encoder:  
  num_strategy: standard  
kmeans:  
  k_min: 2 k_max: 10  
  best_k: null n_init: 10  
  random_state: 42
```

Docker Container

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
RUN mkdir -p output
CMD ["python", "main.py"]

Separazione responsabilita'

main.py — pipeline batch completa
app.py — server Flask + blueprint REST
lib/ — logica riutilizzabile, framework-independent
config.yaml — unico punto di configurazione

Libreria Generalizzata — lib/

Componenti riutilizzabili su qualsiasi dataset tabellare, non solo Seeds

lib/ non e' specifica per Seeds: e' una libreria ML generica. Ogni componente prende un DataFrame in input, lo trasforma e restituisce un DataFrame indipendentemente dal dominio. Per usarla su un nuovo dataset basta cambiare config.yaml.

GenericLoader

Carica da UCI (per ID) o CSV locale. Separa X e y. Missing report e statistiche di base.

GenericEda

Summary statistics, correlazioni Pearson. Identifica coppie altamente correlate ($|r| > 0.85$).

GenericCleaner

Imputa mancanti: mediana/media/moda per numerici, 'unknown' per categorici. Fit su train.

GenericEncoder

Scala feature: StandardScaler, MinMaxScaler o none. One-Hot Encoding per categoriche.

KMeansModel

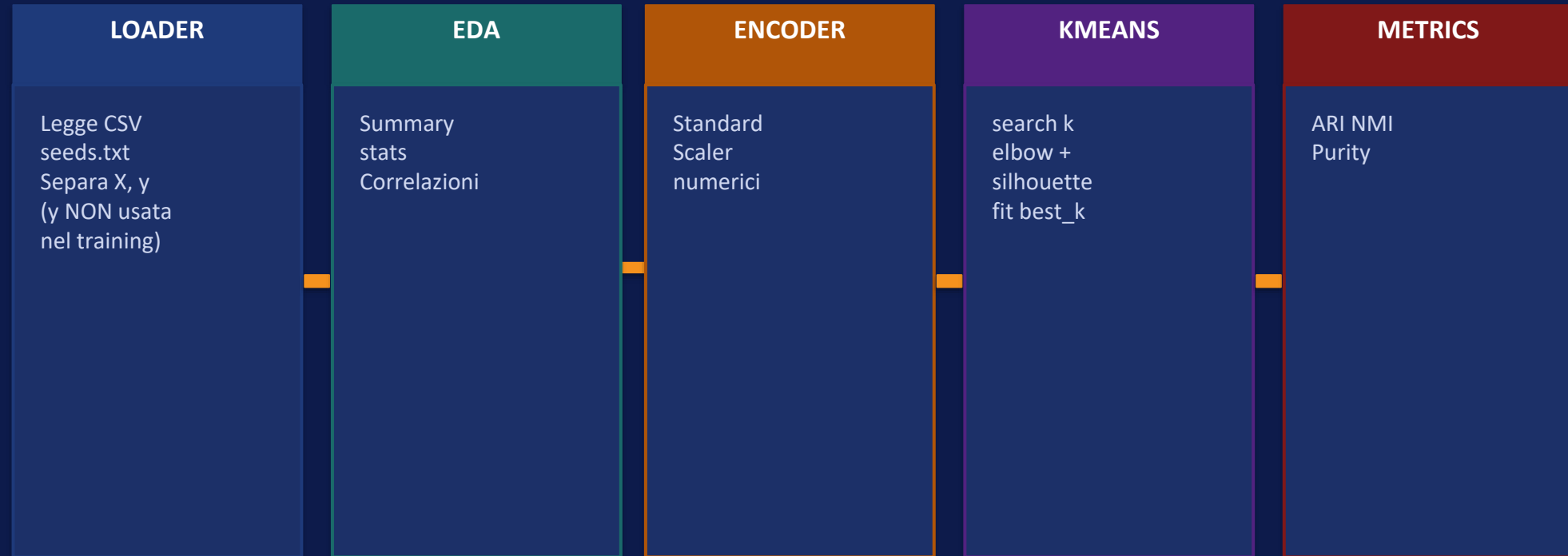
Clustering K-Means: search k (elbow+silhouette), fit, ARI/NMI/Purity. Parametri da config.yaml.

KMeansPlotter

Elbow+Silhouette, PCA 2D scatter, heatmap profili cluster, pairplot. Output PNG su disco.

Data Pipeline

Dal dataset grezzo ai vettori pronti per il clustering



Principio fondamentale — Unsupervised learning: il target (Class) viene caricato ma NON usato durante scaling e clustering. Serve solo per la valutazione finale con ARI/NMI/Purity, che misura quanto i cluster scoperti corrispondono alle varietà reali.

in un caso generale e non accademico di unsupervised learning, la target non è a disposizione, di conseguenza come metrica si usa solo quella interna (inerzia o silhouette).

Algoritmo K-Means

Lloyd's algorithm, inizializzazione k-means++ e convergenza

Obiettivo

Partizionare n punti in k cluster minimizzando l'inertia.

Lloyd's Algorithm — 4 passi

- ① Init — posiziona k centroidi iniziali (casuale o k-means++)
- ② Assign — ogni punto → centroide più vicino (distanza euclidea)
- ③ Update — ogni centroide = media dei punti assegnatogli
- ④ Repeat — ripeti ②③ fino a convergenza (centroidi fermi)

k-means++ (inizializzazione intelligente)

- ① 1° centroide: scelto a caso nel dataset.
- ② Ogni successivo: i punti più lontani dai centroidi già piazzati hanno più probabilità di essere scelti. → I k centroidi partono sparsi nel dataset. → Meno minimi locali, convergenza più rapida.
sklearn: `init='k-means++', n_init=10`

Convergenza

L'algoritmo si ferma quando I centroidi rimangono gli stessi

Ipotesi & Limiti

Assume cluster sferici e di dimensioni simili (distanza euclidea).
→ Non gestisce outlier o forme arbitrarie.

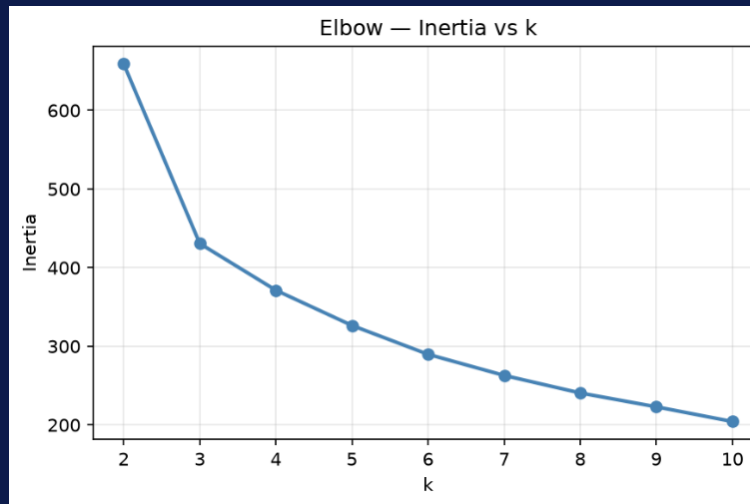
✓ Su Seeds K-Means funziona bene perché le 3 varietà di grano hanno misure geometriche distinte → cluster naturalmente sferici. Confermato Silhouette=0.4007, ARI=0.7733, Purity=0.919.

Scelta di K — Elbow + Silhouette

Due criteri complementari per trovare il numero ottimale di cluster

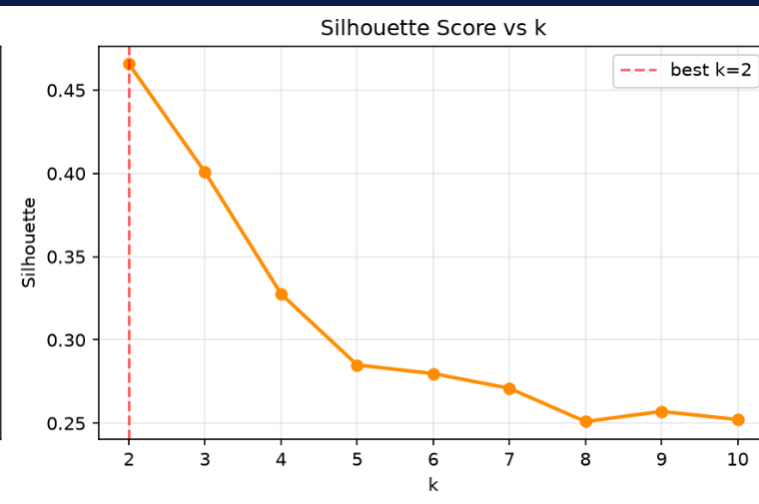
Elbow Method — Inertia

Inertia = somma delle distanze euclidee al quadrato di ogni punto dal proprio centroide. Diminuisce sempre all'aumentare di k . Il 'gomito' è il punto dove la diminuzione rallenta: aggiungere altri cluster porta poco beneficio.



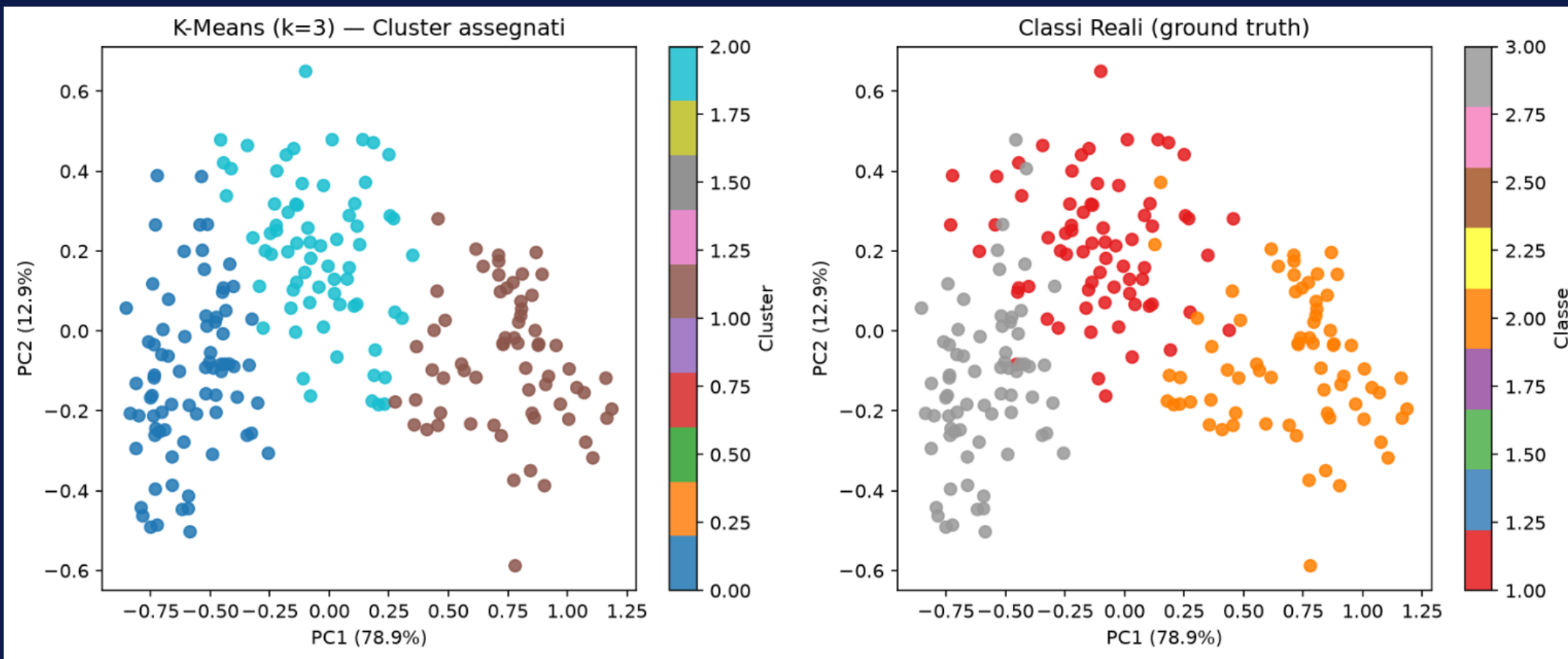
Silhouette Score

L'indice di Silhouette medio indica mediamente quanto i punti sono aggregati correttamente all'interno del cluster, è un indice che va da -1 (i punti sono clusterizzati-male) a 1 (ottima clusterizzazione)



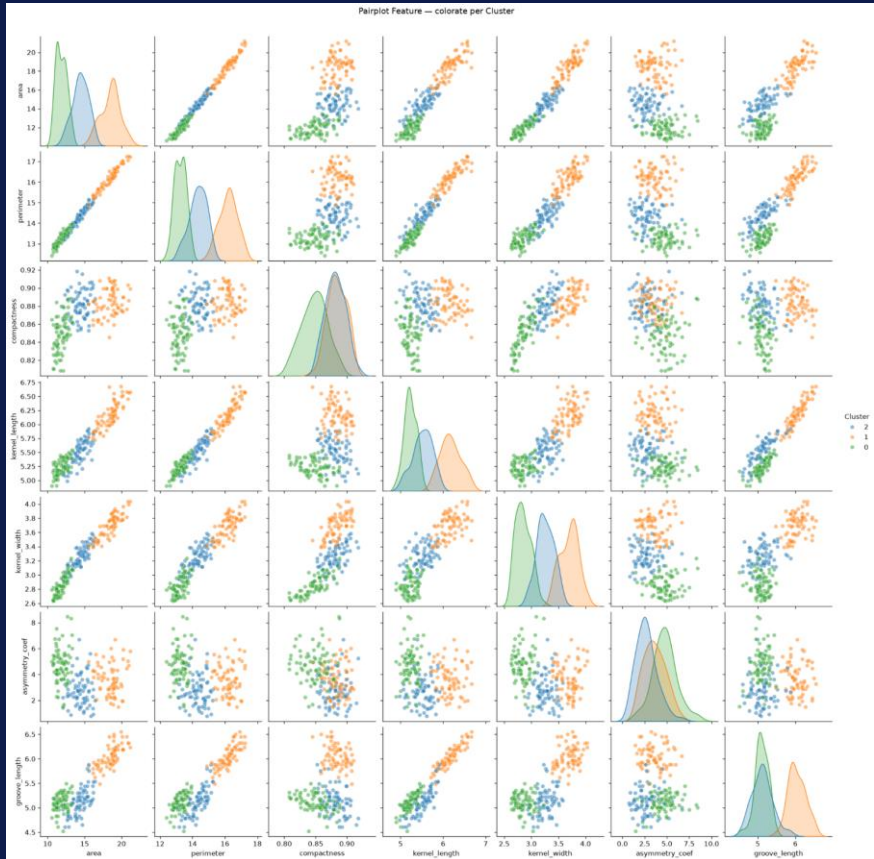
Le scelte possibili valutando in base ai due grafici sono $k=3$ per l'inerzia e $k=2$ per la silhouette, noi abbiamo selezionato $k=3$ in quanto è associato il cambiamento di pendenza del grafico dell'inerzia e inoltre presenta un livello di silhouette molto elevato pari a 0.40

PCA a 2 dimensioni: grafico dei cluster individuati con k=3



I due grafici mostrano rispettivamente i cluster assegnati da K-Means (sinistra) e le varietà reali di grano (destra), entrambi proiettati sulle prime due componenti principali. La forte somiglianza tra le due distribuzioni conferma che K-Means ha individuato struttura reale nei dati: i gruppi scoperti in modo non supervisionato corrispondono quasi perfettamente alle varietà botaniche, senza averle mai viste durante il training.

Statistiche riassuntive dei cluster trovati



Profili Cluster — media feature (scala originale)				
Feature	area	11.86	18.50	14.44
	perimeter	13.25	16.20	14.34
	compactness	0.85	0.88	0.88
	kernel_length	5.23	6.18	5.51
	kernel_width	2.85	3.70	3.26
	asymmetry_coef	4.74	3.63	2.71
	groove_length	5.10	6.04	5.12
	0	1	2	
Cluster				

A sinistra abbiamo lo scatterplot a coppie che mostra la relazione tra tutte le variabili prese due a due, confermando una forte relazione lineare tra alcune coppie di variabili (es. perimetro e area), A destra invece abbiamo la tabella dei profili medi dei cluster sulle variabili originali.

Metriche di Valutazione Clustering

Come misurare la qualità senza e con le etichette vere

Silhouette Score

Metrica interna (non richiede y_true)

Coesione intra-cluster vs separazione inter-cluster.
Range $[-1, +1]$. Valori >0.5 indicano struttura forte.
Usata per la scelta automatica di k .

Seeds: ~0.4

Normalized Mutual Info (NMI)

Metrica esterna (richiede y_true per confronto)

Informazione mutua condivisa tra cluster e etichette, normalizzata in $[0,1]$. Robusta a cluster di dimensioni diverse.

Seeds: ~0.73

Adjusted Rand Index (ARI)

Metrica esterna (richiede y_true per confronto)

Similarità tra la partizione trovata e le etichette reali, corretta per il caso. 1=perfetta, 0=casuale.
Range $[-1, +1]$.

Seeds: ~0.78

Purity

Metrica esterna (richiede y_true per confronto)

Per ogni cluster, prende la classe maggioritaria e somma le assegnazioni corrette.
Semplice e intuitiva.
Range $[0, 1]$.

Seeds: ~0.90

Silhouette disponibile sempre; ARI/NMI/Purity richiedono y_true e vengono calcolate solo nella fase di analisi post-fit, non durante il training.

Configurazione & Automazione

config.yaml come unico punto di controllo | *run_pipeline.py* per automatizzare le chiamate REST

config.yaml

loader:

```
csv_path: data/seeds_dataset.txt
target_col: Class
col_names:
  [area, perimeter, compactness,
   kernel_length, kernel_width,
   asymmetry_coef, groove_length,
   Class]
```

cleaner:

```
num_strategy: median
cat_strategy: unknown
```

encoder:

```
num_strategy: standard
```

kmeans:

```
k_min: 2
k_max: 10
best_k: null
random_state: 42
n_init: 10
```

run_pipeline.py — Automazione

- | | |
|----|--|
| 1 | POST /loader/load
Carica seeds_dataset.txt con col_names |
| 2 | POST /cleaner/configure + fit
Configura e imputa (mediana) |
| 3 | POST /encoder/configure + fit
Configura e scala (StandardScaler) |
| 4 | POST /kmeans/search
Inertia + silhouette per k in [2,10] |
| 5 | POST /kmeans/plot/elbow
PNG elbow method + silhouette |
| 6 | POST /kmeans/fit
Fit con k ottimale (auto-silhouette) |
| 7 | POST /kmeans/plot/pca2d
PNG PCA 2D scatter |
| 8 | POST /kmeans/plot/profiles
PNG heatmap profili cluster |
| 9 | POST /kmeans/plot/pairplot
PNG pairplot feature x cluster |
| 10 | GET /plots/
Lista URL dei PNG generati |

python run_pipeline.py → output/ conterrà i PNG → GET http://localhost:5001/plots/

Risultati — Performance su Seeds

K-Means k=3, StandardScaler, metric=euclidean — valori reali dell'ultima esecuzione

Metrica	Valore	Interpretazione
Silhouette (k=3)	0.4007	Struttura cluster ben definita
ARI	0.7733	Alta corrispondenza con varietà reali
NMI	0.7279	Buona mutua informazione
Purity	0.919	91.9% punti correttamente assegnati
Inertia (k=3)	430.66	Calo netto rispetto a k=2 (659.17)
Cluster 0	72 pt	Semi piccoli e asimmetrici
Cluster 1	67 pt	Semi grandi e allungati
Cluster 2	71 pt	Semi medi e compatti

Silhouette 0.4007

Struttura cluster ben definita. Seeds è un dataset naturalmente separato: le 3 varietà hanno misure geometriche distinte.

ARI 0.7733

L'algorithm recupera quasi perfettamente la suddivisione botanica senza aver mai visto le etichette durante il training.

Purity 91.9%

Solo ~17 punti su 210 vengono assegnati al cluster sbagliato, tipicamente semi di confine tra le varietà.

k=3 confermato

k=3 non è stato scelto perché la silhouette è massima lì, ma perché,il grafico dell'inertia mostra il gomito (cambio di pendenza) a k=3

Visualizzazioni

Quattro grafici per interpretare i cluster — output/ PNG su disco

Elbow + Silhouette

plots.py → *elbow_silhouette()*

Due grafici affiancati: inertia vs k (cerca il gomito) e silhouette vs k (cerca il massimo). Generato dopo POST /kmeans/search. Permette di scegliere visivamente il k migliore.

PCA 2D Scatter

plots.py → *pca2d()*

Riduce le 7 feature a 2 componenti principali (varianza spiegata ~85%). Mostra cluster colorati a sinistra e classi reali a destra: permette di vedere visivamente quanto i cluster corrispondono alle varietà.

Heatmap Profili

plots.py → *profiles()*

Heatmap: cluster (colonne) x feature (righe). Valore = media della feature nel cluster a scala originale. Rivela quali misure geometriche caratterizzano ogni varietà di grano.

Pairplot Feature

plots.py → *pairplot()*

Matrice di scatter plot per ogni coppia di feature, colorata per cluster. Identifica quali coppie separano meglio i cluster. Usa kernel density estimation sulla diagonale.

Sviluppi futuri

Come estendere il progetto con nuovi algoritmi e tecniche

Eliminazione variabili ridondanti

La variabile compactness, essendo una combinazione non lineare di area e perimetro, risulta sostanzialmente costante nei tre cluster (0.85, 0.88, 0.88). Di conseguenza, compactness non apporta informazione discriminante aggiuntiva rispetto alle altre feature e il K-Means l'ha di fatto ignorata come asse di separazione tra i gruppi.

Evita la distorsione delle distanze euclidee

DBSCAN e clustering gerarchico

Nelle analisi successive si possono provare ad implementare altri algoritmi di machine learning non supervisionato, che gestiscono meglio forme di cluster differenti e gli outliers

Vantaggio: gestione outlier, cluster non sferici

Linkage come HP

Aggiungere linkage come iperparametro in config.yaml per AgglomerativeClustering. Combinato con metric: ward/manhattan/cosine, crea uno spazio di iperparametri configurabile come in R.

Obiettivo: flessibilita' pari a R flexclust

Grazie

Seeds Clustering — ML Pipeline con Libreria Generalizzata e API Configurabile

**K-Means
Lloyd**

Algoritmo
partizionale

**Elbow +
Silhouette**

Scelta di k

**ARI NMI
Purity**

Valutazione cluster

**config
.yaml**

Configurazione

**run
pipeline**

Automazione API

**REST API
Flask**

Architettura

**Docker
/plots/**

Deploy + grafici

**PCA 2D
Pairplot**

Visualizzazioni